Analysis of Algorithm

Time Complexity

Space Complexity

Big O Notation.

Constant time Complexity algo ⇒ algo takes time independent of input size.

Find sum of 3 numbers

① Read 3 numbers (no1, no2, no3) — 1

② Sum = no1 + no2 + no3 — 1

③ Print Sum — 1

④ Stop.

Total = 3

If its a constant then time complexity = O(1)

Find sum of N numbers

(u - l + 1) ← [l, u]

[l, u)

1. Read N — 1
2. Create space for N numbers (nums) — 1
3. for i = 0 to (n-1) ⇒ (n-1) - 0 + 1 = n
   - 3.1 Read nums[i] — 1 — n times
4. Sum = 0 — 1
5. for i = 0 to (n-1)
   - 5.1 Sum = Sum + nums[i] — 1 — n times
6. Print sum — 1
7. Stop

Total = ~~4~~ + ~~2~~n

① Ignore constants

⇒ O(n) ← linear time complexity.

Find time complexity of following

```
for i = 0 to (n-1)
    for j = (i+1) to (n-1)
        ... do something ...
```

Sum of first n natural numbers $= \frac{n(n+1)}{2}$

i	j	how many times inner loop runs?
0	1 2 .. (n-1)	(n-1) - 1 + 1 = n-1
1	2 3 ... (n-1)	(n-1) - 2 + 1 = n-2
2	3 4 ... (n-1)	n-3
⋮	⋮	⋮
(n-2)	(n-1)	1
(n-1)	X	

Total = $1 + 2 + \dots + (n-3) + (n-2) + (n-1)$

$$= \frac{(n-1)(n-1+1)}{2} = \frac{(n-1)(n)}{2}$$

$$= \frac{1}{2}(n^2 - n)$$

As value of n gets large $n^2 - n \approx n^2$

① Ignore constants $\Rightarrow n^2 - n$

② Pick term with highest power of n.

$\Rightarrow O(n^2)$ ← quadratic time complexity.

Find time complexity of

```
for i = 0 to (n/2)
    for j = (i+1) to (n-1)
        for k = 0 to (m-1)
            ... do Something ...
```

Stack

- Stack is a linear data structure.
- Stack is a container of objects.

Stack operations

- LIFO – Last In First Out
- Elements are added and removed according to LIFO principle.
- Operations are performed with respect to “**top**” of stack.

Stack as Abstract Data Type (ADT)

- push ← add element to stack
- pop ← remove element from stack

peek

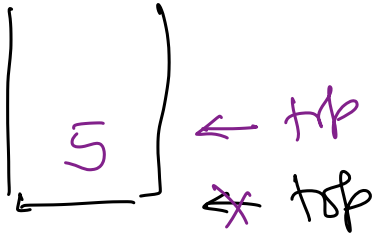
isEmpty

isFull

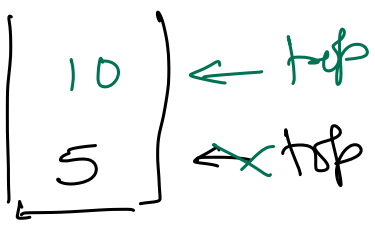
```java
public interface Stack {
    void push(int element);
    int pop();
    int peek();
    boolean isEmpty();
    boolean isFull();
}
```

Stack Operations

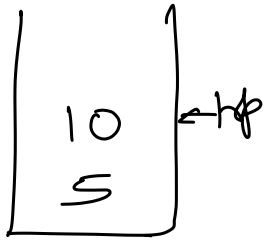
Empty stack

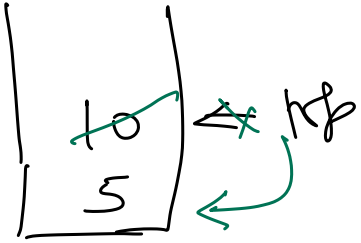


push(5)

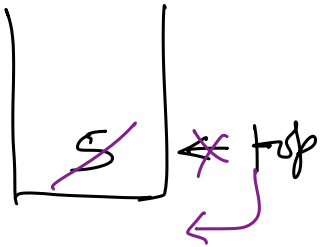


push(10)



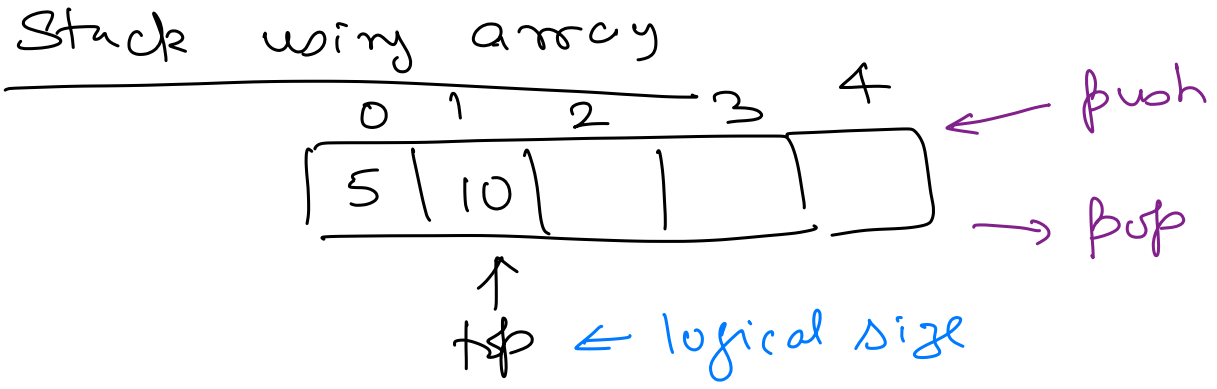


pop() => 10



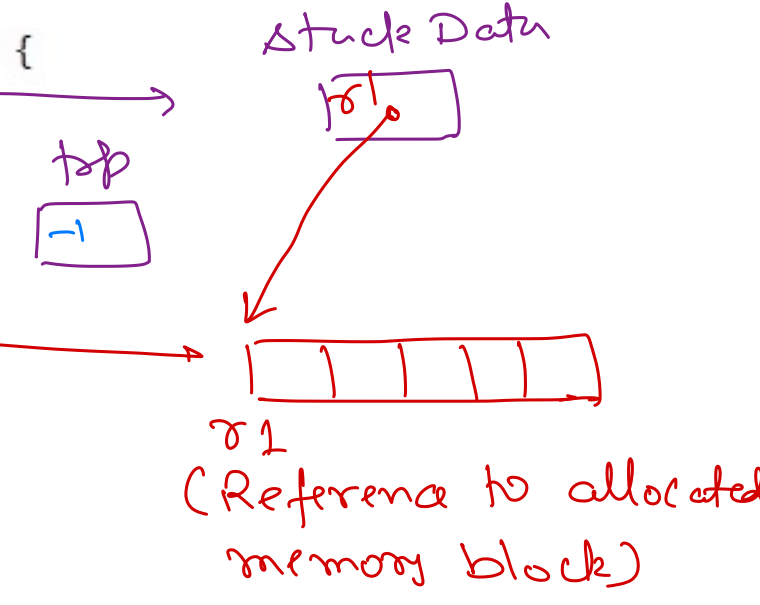
pop() => 5

Empty stack




```java
public class FixedSizeStack implements Stack {
    private int[] stackData;
    private int top;

    public FixedSizeStack(int n) {
        stackData = new int[n];
        top = -1;
    }
```



Push(element)
- If stack is full then stop.
- Make space at top for new element.
- Store new element and make it topmost element.

```
if (isFull())
    if (top == (stackData.length - 1)) return;
    top = top + 1; // ++top;
    stackData[top] = element;
```

Pop()
- If stack is empty then stop.
- Set topmost element as result.
- Remove topmost element and make element below top, the topmost element.
- Return the result.

```
if (isEmpty())
    if (top == -1) return -1;   → throw appropriate exception
    result = stackData[top];
    --top;
    return result;
```

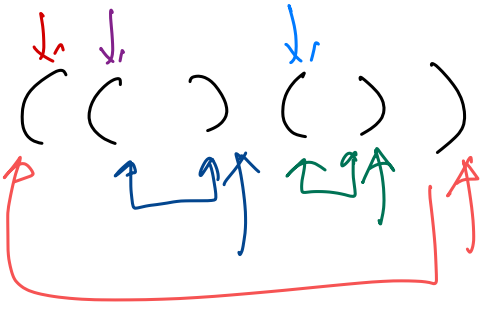
IsEmpty()
- If no element stored at top then return true.
Else return false

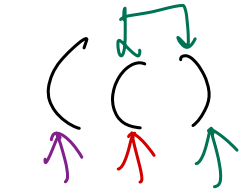
```
if (top == -1) return true;
return false;
```

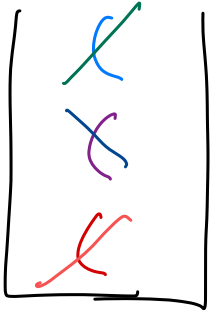
IsFull()
- If no space left for new element to be stored then return true.
Else return false.

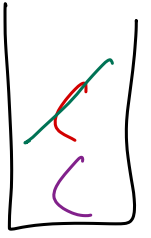
```
if (top == (stackData.length - 1))
    return true;
return false;
```

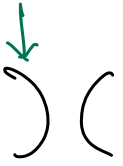
Check if string of parenthesis is balanced or not.

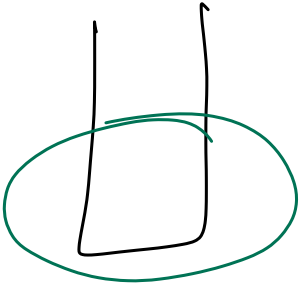








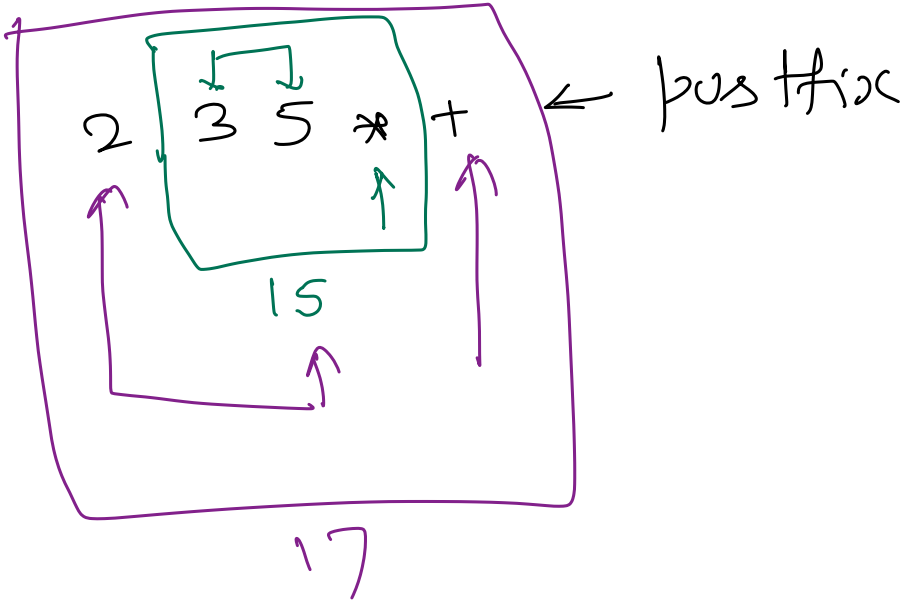


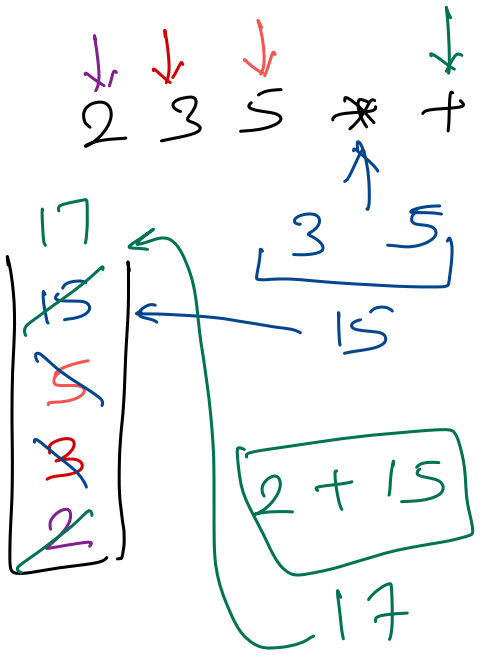


Evaluate postfix expression

2 + 3 * 5 ← infix

+ 2 * 3 5 ← prefix





Min Stack
Max Stack

Implement 2 stacks in an array

Implement n - stacks in an array

Application of stack

1. O.S. for function calls.
2. Recursive to Iterative algo.
3. Other algorithms.